

INVENTION DISCLOSURE — REVISED

Agent-Instantiated Persistent Context Services (AIPCS)

A Pattern for Autonomous, Domain-Adaptive Memory Infrastructure via MCP

Inventor	Randall Mark
Date of Disclosure	04 May 2026
Field	Artificial Intelligence / Agent Architecture / Knowledge Management
Repository	https://github.com/randallcmark/application_tracker
Proving Ground	Application Tracker — self-hosted career management platform
Status	Pre-publication invention disclosure — all rights reserved by inventor
Intent	Open attribution via academic preprint (arXiv). Not seeking commercial restriction.

1. Field of the Invention

This invention relates to memory architectures for Large Language Model (LLM)-based AI agents. Specifically, it describes a pattern — and reference implementation — by which an AI agent autonomously designs, scaffolds, and registers a domain-specific persistent data service in response to a stateful tracking problem encountered at runtime. The service is exposed as a Model Context Protocol (MCP) tool, making it queryable and persistent across sessions and across MCP-compatible AI clients.

2. Background and Problem Statement

2.1 The Statelessness Problem

LLM-based AI agents are fundamentally stateless. Each session begins without knowledge of prior interactions. For simple queries this is acceptable; for complex, multi-entity, evolving domains — job searching, medical management, project tracking, legal case management — it is a critical limitation. Users compensate by manually re-establishing context, a burden that grows with domain complexity.

2.2 Existing Approaches and Their Limitations

Several approaches address agent memory, each with significant limitations:

- Pre-designed schemas with semantic retrieval (mem0, mcp-memory-service, Hindsight): The developer pre-defines the memory schema; the agent only populates

and retrieves. No agent authorship of structure. Not adaptable to novel domains without developer intervention.

- File-based self-organisation (MemFabric): The agent organises plain markdown files with descriptive filenames. Self-organising, but unstructured — not queryable relationally. Suitable for recall but not for structured domain queries.
- Parametric memory: Model fine-tuning to embed knowledge. Expensive, inflexible, not accessible as a tool to other agents or clients.
- Academic adaptive schemas (PISA, Nemori): Cognitively-inspired architectures for dynamic memory organisation. Related in spirit but not deployed as composable, MCP-integrated, consumer-accessible service patterns.
- MemGPT / Letta: OS-inspired memory paging between context window and external store. The external store schema is pre-defined, not agent-designed.

2.3 The API Access Friction Problem

A secondary motivation for this invention is access equity. AI-assisted features in productivity tools typically require developer API subscriptions — a separate paid tier beyond consumer AI subscriptions. For users who already hold a Claude or ChatGPT subscription, this creates unnecessary duplication of cost. This is especially acute for lower-income users, such as job seekers, who are most in need of AI assistance but least able to pay for multiple subscriptions.

The MCP protocol, combined with OAuth 2.0 Dynamic Client Registration (DCR), offers a standards-based path: users bring their existing consumer AI client as the reasoning engine, connecting it to a self-hosted service that acts as system of record. This invention incorporates this model as a design constraint.

2.4 The Unsolved Problem

No existing approach enables an AI agent to: (a) encounter an arbitrary complex tracking domain at runtime, (b) autonomously design an appropriate relational schema for it, (c) scaffold a live queryable service around that schema, (d) register that service as a reusable MCP tool — persistently accessible across sessions and across different AI frontends — and (e) make that service accessible to consumer AI users without requiring a separate developer API subscription.

3. Summary of the Invention

Agent-Instantiated Persistent Context Services (AIPCS) is a pattern and reference implementation. When an AI agent determines that a user's task requires structured, persistent, queryable memory that does not yet exist, the agent itself designs and deploys that memory infrastructure on demand — then registers it as an MCP tool accessible to any compatible AI client.

The pattern has five defining characteristics:

- **Agent-as-architect:** The agent analyses the domain and designs the relational schema — rather than consuming a pre-designed one.
- **Structured and queryable:** The resulting store is a proper relational database (e.g., SQLite), enabling precise structured queries — not a flat file or vector index.
- **Service-wrapped:** The schema is wrapped in a lightweight API (e.g., FastAPI), making it a live callable service that persists independently of any AI session.
- **MCP-registered:** The service is exposed as an MCP tool, discoverable and usable by any MCP-compatible AI client across future sessions without re-instantiation.
- **Consumer-accessible via DCR:** The service uses OAuth 2.0 with Dynamic Client Registration so users can connect their existing consumer AI subscription as the reasoning engine, without requiring a separate developer API key.

4. Detailed Description

4.1 The Core Pattern — Step by Step

Step 1: Problem Recognition

The agent identifies that the user's task has sufficient structural complexity and persistence requirement to warrant a dedicated memory service. Triggers include: multi-entity tracking across time, relational dependencies between domain concepts, the need for structured queries, or explicit user expression of a tracking need.

Step 2: Domain Analysis and Schema Design

The agent reasons about the domain — identifying entities, attributes, relationships, and likely query patterns. It generates a relational schema. This schema design is the primary novel act: the agent performs a data modelling task, not merely populating a pre-existing model. The schema may include normalised tables, foreign key relationships, index hints for anticipated query patterns, and field-level type and constraint definitions.

Step 3: Service Scaffolding

The agent generates and deploys: (a) a database initialisation script creating the schema, (b) a lightweight API layer (FastAPI or equivalent) exposing CRUD and query operations, and (c) a service runner making the API available at a known local endpoint. This is deployable via Docker for privacy-first local hosting.

Step 4: MCP Tool Registration

The agent registers the new service as an MCP tool — generating a tool definition including name, description, parameter schema, and endpoint. Tool naming follows a workflow-oriented taxonomy (`domain_object_action`) rather than a database-oriented one, a deliberate design principle ensuring tools are discoverable and meaningful to agent reasoning. The registration persists across sessions.

Step 5: Ongoing Use and Evolution

Subsequent sessions use the registered MCP tool to read and write structured data. The schema may evolve — the agent can extend it as domain understanding deepens, using migration patterns to preserve existing data. The agent manages its own schema evolution without developer intervention.

4.2 The Tool Taxonomy Contribution

A secondary but significant design contribution is the principle that MCP tools should be workflow-oriented and domain-language-driven, not database-oriented. Most MCP implementations expose tools that map directly to database operations (`get_record`, `update_field`). This invention explicitly rejects that approach in favour of tools that map to meaningful agent-facing workflow operations:

- Tool names follow `domain_object_action` (e.g., `job_context_get`, `ai_output_create`)
- Tools represent workflow operations, not database rows
- Tool descriptions are constrained and agent-discoverable
- Read and write operations are explicitly separated
- Generation and persistence are separated — the agent reasons, the service stores

This taxonomy principle is generalisable: any AIPCS-instantiated service should expose workflow-oriented tools, not raw data access, to preserve the integrity of the domain model as the system of record.

4.3 Authentication and Consumer Access Model

The pattern incorporates OAuth 2.0 with Dynamic Client Registration as the authentication model. This enables a consumer AI user (e.g., a Claude.ai or ChatGPT subscriber) to connect their existing AI client to a self-hosted AIPCS instance using a standards-based registration and consent flow — without requiring a separate developer API key or paid API subscription.

The authorization model includes: dynamic client registration, Authorization Code with PKCE for user-facing clients, scoped access tokens, token expiry and revocation, consent records with domain-language descriptions, client management UI, and audit logging of all MCP-created outputs. The self-hosted deployment model means user data remains private and locally controlled.

4.4 Security and Provenance Model

Every AIPCS-created output records provenance: `created_via`, `client_id`, `tool_name`, `tool_version`, `owner_id`, `source_context`, `input_summary`, and `created_at`. MCP is disabled by default. Write and mutation tools are strictly separated from read tools. A confirmation model (`propose/confirm`) is used for state-changing operations. Prompt injection from external content is mitigated by treating ingested content as labelled data, not instruction.

4.5 Deployment Model

The reference implementation uses Docker Compose, with a sidecar MCP service alongside the main application. This enables privacy-first local deployment suitable for home servers (e.g., QNAP NAS, Raspberry Pi, local machine). The pattern is equally deployable in cloud environments for users who prefer hosted operation.

4.6 Generalisability

While conceived in the context of a career management application, AIPCS is explicitly domain-agnostic. Any sufficiently complex stateful tracking problem is a candidate: medical history, project management, financial tracking, research notes, legal case management, habit tracking, and beyond. The agent selects the appropriate schema for the domain at instantiation time. The pattern is a universal primitive, not a fixed application schema.

5. Claims of Novelty

The following are asserted as novel contributions:

- **Claim 1 — Agent-designed relational schema:** The AI agent, rather than a developer, designs the relational schema for a persistent context service in response to a domain encountered at runtime.
- **Claim 2 — On-demand service scaffolding:** The agent generates and deploys a complete API service (relational database + HTTP layer) around its self-designed schema, without pre-configuration by a developer.
- **Claim 3 — MCP registration of agent-generated services:** The agent registers its newly created service as an MCP tool, making it discoverable and usable by any MCP-compatible AI client across sessions.
- **Claim 4 — Consumer AI access via DCR:** The pattern enables consumer AI users to access agent-generated persistent context services using their existing AI subscriptions via OAuth 2.0 with Dynamic Client Registration, without requiring a separate developer API key.
- **Claim 5 — Domain-agnostic instantiation:** The pattern is a universal primitive — instantiable for any domain requiring structured, persistent, queryable agent memory — not a fixed application schema.
- **Claim 6 — Composability across AI frontends:** Because the service is MCP-exposed, it is accessible by multiple different AI clients without duplication of the underlying data or service.
- **Claim 7 — Workflow-oriented tool taxonomy:** MCP tools generated by AIPCS follow a workflow-oriented, domain-language-driven taxonomy (domain_object_action) rather than a database-oriented one, preserving the integrity of the domain model and making tools agent-discoverable.

- **Claim 8 — Agent-managed schema evolution:** The agent manages the evolution of its own schema over time — extending, migrating, and adapting the data model as domain understanding deepens — without developer intervention.

6. Relation to Prior Art

The inventor conducted a review of relevant prior art. The following are acknowledged as related but distinct:

- **MemFabric (MrBoor, 2025):** LLM-managed self-organising markdown file memory. Related in spirit (agent manages its own storage structure) but produces flat files, not structured queryable schemas, and does not scaffold MCP-registered services.
- **PISA (2024):** Piaget-inspired adaptive memory schema architecture. Conceptually the closest academic antecedent — agent-influenced schema evolution — but is an academic architecture, not a deployable MCP-integrated service pattern.
- **Nemori (2025):** Self-organising Memory-Augmented Generation. Addresses retrieval from conversational memory, not schema design or service scaffolding.
- **MemGPT / Letta:** OS-inspired memory paging. The external store schema is pre-defined by developers, not designed by the agent.
- **mcp-memory-service / Hindsight:** MCP-connected persistent memory backends with pre-defined developer schemas. The agent populates but does not architect the structure.
- **SchemaAgent (2025):** LLM multi-agent framework for database schema generation from requirements. Generates schemas for existing data problems — not as a universal memory primitive or MCP-integrated service pattern with consumer DCR access.

None of the reviewed prior art combines agent-designed schemas, on-demand service scaffolding, MCP tool registration, workflow-oriented tool taxonomy, DCR-based consumer AI access, and domain-agnostic instantiation into a unified deployable pattern.

7. Origin and Development Context

This invention arose from the development of Application Tracker (github.com/randallcmark/application_tracker), a self-hosted Docker-deployable career management platform. The application includes job listing management, application pipeline tracking (kanban-style), AI-assisted artefact generation (CVs, cover letters, STAR competency evidence), browser plugin and email ingestion, focus task management, and an inbox for newly ingested items.

In designing the MCP integration for Application Tracker, the inventor identified the API access friction problem: users who already held consumer AI subscriptions faced a separate cost barrier to access AI features in the application. The DCR-based OAuth model was designed so users could bring their existing Claude or ChatGPT subscription as the AI reasoning engine, with Application Tracker acting as the system of record.

From this specific solution, the inventor recognised a generalised pattern: any time an AI agent encounters a sufficiently complex tracking problem, it could instantiate its own structured memory infrastructure rather than relying on pre-built stores. Application Tracker is the first proving ground for AIPCS, but the pattern is designed to be extracted and published as a general-purpose contribution.

The inventor also notes relevant prior personal work: an earlier agent memory layer that was explicitly designed and defined by the developer rather than by the agent. That experience directly motivated the present invention, which inverts the relationship: the agent, not the developer, is the architect of its own memory structure.

8. Intent and Licensing

The inventor does not seek to commercially restrict this invention. The intent is attribution and open contribution. The inventor intends to publish as a technical preprint (arXiv) to establish public prior art under the inventor's name, and to release a reference implementation as open-source software under an attribution-friendly licence.

This document serves as a timestamped invention disclosure establishing the inventor's conception of AIPCS prior to publication. All rights are reserved by the inventor pending that publication.

9. Proposed Next Steps

- Finalise the AIPCS Pattern Specification as an implementation-agnostic document.
- Build the reference implementation within Application Tracker as the proving ground.
- Extract the generalised AIPCS framework from the application-specific implementation.
- Write and submit a technical preprint to arXiv establishing public prior art.
- Release the reference implementation as open-source under an attribution-friendly licence.

Inventor Declaration

I, Randall Mark, declare that I am the original inventor of the Agent-Instantiated Persistent Context Services (AIPCS) pattern as described in this document, that this disclosure represents my genuine conception of this invention, and that to my knowledge no prior art combines these elements in the manner described herein.

Date: 04 May 2026

Signature: 

Randall Mark